Syll: An Open-Source Multimodal Agent Harness for Teachable Personal Automation

Bo Zhang[1*] Borui Zhang[1*†] Chenghao Jiang[1*] Minglei Shi[1*]
Xiaofeng Wang[2] Zheng Zhu[2] Jie Zhou[1] Jiwen Lu[1#]
[1]Tsinghua University, [2]GigaAI
[*]Equal contribution. [†]Project Leader. [#]Corresponding author.

Project page: https://github.com/THU-SAGE/syll

Abstract. Personal AI agents must increasingly operate across APIs, shells, web surfaces, and desktop GUIs, yet many systems remain tuned to a single interface and offer limited support for user teaching and auditability. We present **Syll**, an open-source, self-hosted multimodal agent harness that unifies MCP/API tools, CLI execution, and visual GUI control in a modular runtime, enabling agents to coordinate computer use across heterogeneous interfaces while streamlining how users and agents exchange information. At the core of Syll is a bidirectional user–agent interaction layer: users teach procedures through direct demonstration, which Syll compiles into reusable skills; agent execution is translated back into multimodal evidence—logs, keyframes, and approval checkpoints—for inspection and control. Syll further externalizes memory, skills, routines, and governance as editable local artifacts, supporting straightforward inspection, extension, and downstream development. We report mechanism-oriented studies that validate multimodal routing, teachable GUI replay, and persistent local artifacts. We hope Syll can serve as a practical open-source foundation for personal automation that users can teach, inspect, and continuously extend.

1 Introduction

Personal AI agents are increasingly asked to complete tasks in the real world rather than answer isolated questions. A single request may require retrieving local materials, running a script, checking a web dashboard, and operating a closed-source desktop application whose state is visible only on screen. In such settings, strong language modeling is necessary but insufficient. Reliable task completion depends on the surrounding **agent harness**: the layer that constructs context, selects interfaces, coordinates execution, and returns results in a form that users can review. Recent evidence suggests that harness design and agent-computer interface choice materially affect downstream performance [10, 16]. This motivates a systems perspective on personal automation.

A first unresolved question is how one agent should act across heterogeneous work surfaces. **MCP/API-style tools** are reliable when clean machine interfaces already exist [9, 13]. **CLI** execution is powerful for composable local control and long-horizon deterministic workflows [16]. Yet many practical tasks still depend on **GUI** interaction because state is only visible in pixels, APIs are unavailable, or users require screen-level infomation [12, 15, 19, 20]. Existing systems often privilege one interface family, forcing an avoidable trade-off between reliability and coverage. For personal automation, however, these surfaces are complementary: the right system should route each step through the narrowest interface that can complete the task while preserving the required audit surface.

A second question is how knowledge should flow between users and agents. Many current systems assume that operational knowledge must first be rewritten as prompts, schemas, or skill files before the agent can use it. This assumption is too restrictive. Many users know *how* to perform a task, but do not know how to formalize that procedure

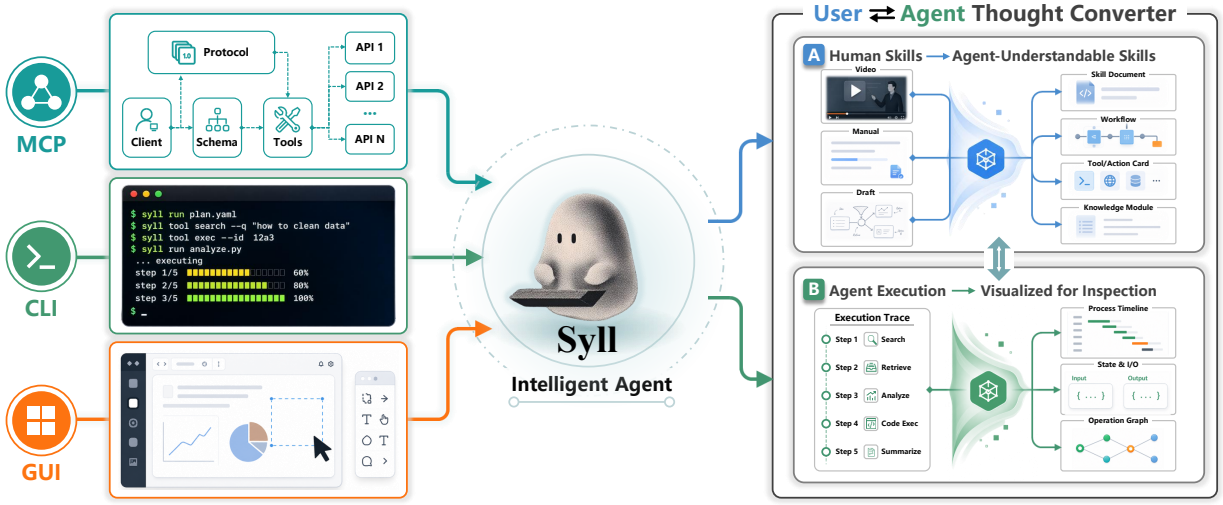


Figure 1. Syll overview. The system unifies three execution surfaces—MCP, CLI, and GUI—around a central multimodal agent harness, while the right branch visualizes the bidirectional user-agent thought converter that turns demonstrations into reusable skills and execution traces into reviewable multimodal evidence.

into a reusable representation. Conversely, when an agent completes a long-horizon task, a raw text trace is often too costly to inspect. We therefore argue that personal automation needs a **bidirectional user-agent thought converter**: human know-how should be converted into reusable machine-executable skills, and agent execution should be converted back into multimodal evidence such as screenshots, keyframes, logs, diffs, previews, and approval checkpoints.

We instantiate this view in **Syll**, an **open-source**, self-hosted multimodal agent harness for **teachable personal automation** shown in Figure 1. Syll unifies MCP/API tools, CLI execution, and visual GUI control within one modular runtime. At its core, users can teach procedures through direct demonstration, which Syll converts into reusable skills, while the runtime translates execution back into multimodal evidence for efficient review. Beyond execution, Syll externalizes memory, skills, routines, governance, and traces as **persistent local artifacts**, making the system inspectable for users and straightforward to extend for developers.

This design is motivated by both usability and research value. Keeping execution surfaces, skill conversion, evidence generation, channels, context construction, and confirmation gates explicit reduces cross-component entanglement and lowers the barrier to secondary development. It also improves auditability: as agent task horizons continue to grow [7], and as open-ended computer-use settings continue to expose substantial verification burden [1], practical usefulness depends not only on model intelligence but also on whether the system is **teachable**, **auditable**, and **continuously extensible**. We therefore evaluate Syll through mechanism-oriented studies aligned with these goals.

This technical report makes the following contributions:

- **Open-source multimodal agent.** We present Syll as an open-source multimodal agent harness that supports MCP/API, CLI, and GUI as complementary execution surfaces and routes tasks to the most appropriate action layer.
- **A bidirectional user-agent thought converter.** We design a teachable workflow that turns direct user demonstrations into reusable skills and an evidence pipeline that translates agent execution back into reviewable outputs.
- **Persistent local artifacts and modular design.** We externalize memory, skills, routines, traces, evidence, and governance as user-editable local artifacts within a modular architecture, making the system extensible.
- **Mechanism-oriented evaluation.** We validate Syll through studies targeted at three core system properties: multimodal routing, teachable GUI replay, and persistent local artifacts.

Table 1. High-level comparison of agent system families. "Partial" means the capability exists but is not the primary design center.

System family	GUI	CLI/API	Demonstration	User memory	Main audience
CLI-only coding agents	No	Yes	No	Partial	developers
MCP/tool frameworks	No	Yes	No	Partial	developers/integrators
GUI agents	Yes	Partial	Partial	Partial	automation builders
Record-replay/RPA tools	Yes	Partial	Yes	No	operations teams
Companion chat agents	No	Partial	No	Yes	end users
Syll	Yes	Yes	Yes	Yes	users and developers

2 Related Work

2.1 Agent-Centric Task Execution

Recent progress in autonomous agents has concentrated on strengthening reasoning, planning, and perception. Foundational frameworks such as ReAct [17] and Reflexion [14] introduced iterative reasoning-action loops that allow language models to decompose tasks, interact with environments, and refine behavior through textual feedback. Subsequent work extended these paradigms to tool invocation and long-horizon execution, enabling agents to call external APIs and structured functions [13, 16]. With the emergence of vision-language models, the interaction surface has broadened to graphical user interfaces. Systems like Claude Computer Use [2], Operator [10], and UI-TARS [12] perceive and act upon GUIs through screenshots and low-level control primitives. Despite these advances, agent-centric approaches typically treat the execution surface as a fixed given, an API, a browser, or a specific GUI environment, and concentrate on what the agent decides to do. Consequently, interaction knowledge, such as which pixels to click, which states to wait for, and what outcomes to expect, remains confined within a single trajectory transcript and is rarely externalized as a reusable, auditable artifact that can be repurposed by the user or the system.

2.2 Harness-Centric and Record-and-Replay Systems

Complementing the advances in agent intelligence,, a line of work targets the system infrastructure that supports reliable, locally owned agent execution. Frameworks such as OpenClaw [11] and NanoBot [4] provide local-first runtimes that integrate persistent memory, tool execution, scheduling, and multi-channel communication. The Model Context Protocol (MCP) [9] standardizes how agents interact with external tools through structured, schema-driven APIs. These harness-centric systems offer strong modularity and extensibility but are predominantly confined to textual or structured modalities; GUI interaction, when present, is often treated as a separate subsystem rather than a unified modality.

Record-and-replay tools and programming-by-demonstration (PbD) systems address a complementary need: enabling users to automate repetitive GUI workflows without scripting. Sikuli [18] uses screenshot-driven search to replay visual procedures, CoScripter [8] captures step-by-step action descriptions, and Eager [3] proactively detects iterative patterns. More recent efforts like ShowUI-Aloha [19] incorporate human-taught GUI trajectories for agent learning. These approaches excel at capturing what the user does, but they typically lack a semantic phase structure, a persistent skill registry shared with non-GUI tools, and an evidence pipeline that translates execution back into inspectable artifacts. As a result, a recorded workflow often remains a brittle, isolated macro rather than a first-class runtime abstraction that can be retrieved, scheduled, verified, and refined alongside other skills.

2.3 Teachable Automation and Execution Audit

Two design goals that are central to Syll, allowing users to *teach* procedures through demonstration and providing *audit evidence* for every executed action, remain underrepresented in the current agent landscape. On the teaching side, existing PbD and record-and-replay tools show that end-user demonstration is a viable way to transfer operational knowledge, but they do not connect this knowledge to a general-purpose agent's context construction or to a shared skill registry that also includes MCP tools and CLI commands. On the audit side, agent evaluation has largely been success-rate driven; benchmarks like OSWorld [15], WebArena [20], and OSWorld-Human [1] measure task completion, yet few systems explicitly structure execution traces into layer-specific, user-reviewable evidence artifacts such as keyframes, diffs, semantic expectations, and approval records. Syll's bidirectional thought converter treats teaching and auditing as two sides of the same artifact lifecycle: a user demonstration is packaged as an inspectable skill with embedded

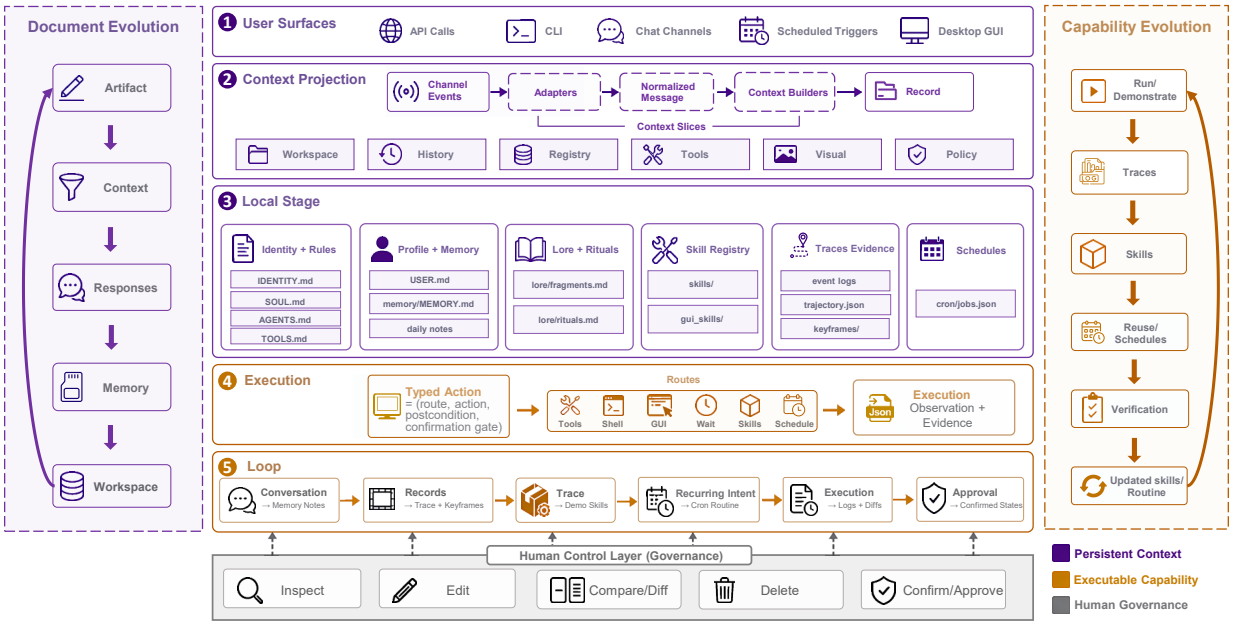


Figure 2. Syll runtime overview. Requests enter one local runtime, are grounded in shared workspace context, routed through tools, commands, or GUI control, and leave audit evidence as confirmations, logs, screenshots, keyframes, and workspace diffs.

visual cues and post-state expectations, while runtime execution—whether over MCP, CLI, or GUI—generates a corresponding audit trail that satisfies a minimum coverage constraint, making the agent's observation-decision-action loop of the agent transparent long after the task finishes.

2.4 From Fragmented Capabilities to a Unified Artifact Lifecycle

Taken together, prior work reveals a structural gap: no existing system jointly resolves multi-surface execution, user-teaching workflows, and persistent, user-inspectable execution evidence within a single framework. Table 1 summarizes this landscape qualitatively: GUI agents provide visual control but lack demonstration handling; MCP/tool frameworks offer structured APIs but no GUI or teaching primitives; record-replay tools support demonstration but omit cross-surface routing and audit; companion chat agents emphasize user memory but are not designed for GUI automation or audit trails. Syll addresses this gap by elevating execution surfaces, skill registration, evidence generation, and memory artifacts into a unified, modular runtime. The resulting system treats computer-use as an artifact lifecycle, from demonstration to skill, from execution to audit trail, and externalizes all mutable state as editable local documents, enabling users to teach, inspect, and continuously extend their personal automation.

3 Method

Syll is an open-source, self-hosted computer-use harness for multimodal personal automation. It is designed for the practical setting in which a user moves between a phone, a browser, a terminal, local files, scheduled routines, and desktop applications, while expecting one local agent to preserve context and act across these surfaces. We model the runtime as an agent loop. At each turn the executor runs a typed action through one of the layered tool surfaces (structured APIs, the CLI, or visual GUI control). The context builder feeds it the joint external-plus-artifact state, and a verification step (success checks, audit evidence, approval gates) decides whether to commit, retry, or wait for explicit user approval. The cross-surface runtime layout is documented in Section F, the demonstration-skill schema in Section D, and the formal artifact-option model in Section C.

3.1 Multimodal Execution Space

Syll targets recurring personal workflows that span heterogeneous surfaces, such as initiating a task in chat, manipulating local files, and confirming actions on mobile. Single-interface agents force an avoidable trade-off between reliability and coverage. Syll instead treats the diverse surfaces as one complementary execution space and routes each request through the narrowest viable layer. Schema-driven tools and standardized connectors, including MCP, are preferred when an application exposes a clean machine interface. When no such API exists but the target state remains textually accessible, the CLI provides reproducible control through shell, scripts, and filesystem operations. When the state is only visible in pixels, or the user needs screen-level evidence, Syll reasons over screenshots and dispatches low-level desktop events. This fallback keeps legacy applications and human-facing workflows reachable without custom connectors.

Table 2. Execution-layer routing policy. Syll selects the narrowest route that can complete the task while preserving the audit surface required by policy or by the user.

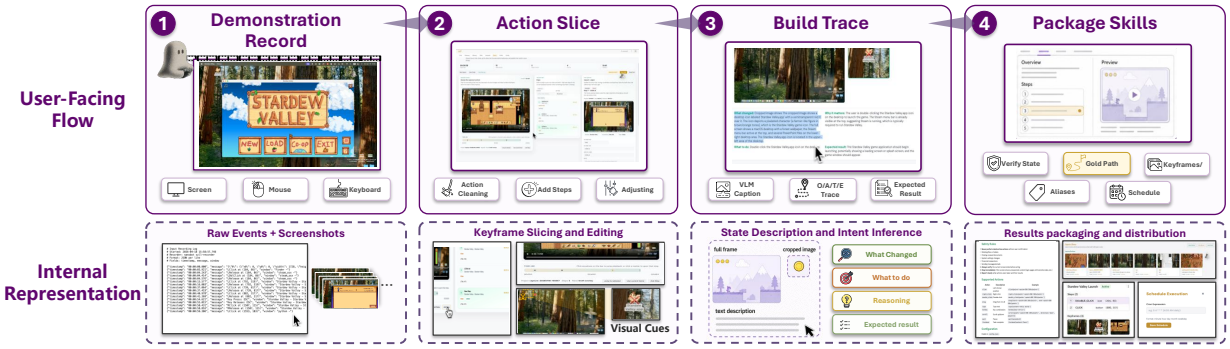
Route	Selected when	Evidence retained
Structured tools/API	A schema-described operation, connector, file tool, delivery tool, or configuration route exists	Tool name, validated arguments, output/error, confirmation record
Local command/resource	Desired state is machine-readable or reproducible through shell, scripts, local files, or browser-accessible resources	Command, working directory, stdout/stderr, generated files or diffs
Visual GUI control	State is only visible in pixels, no reliable API exists, or the user needs screen-level evidence	Screenshots, keyframes, low-level actions, visual observations, semantic expectations
Confirmation gate	External delivery, destructive local change, account change, purchase, or scheduled send would create a user-visible side effect	Proposal, candidate artifact, approval token, final side-effect record

The route choice also determines what the verification step retains (Table 2). A structured tool call yields validated arguments and machine-readable results, the CLI preserves commands, working directories, and textual outputs, GUI control retains screenshots, keyframes, and action traces, and side-effectful operations such as file delivery, account changes, scheduled sends, and destructive local writes are split into proposal and execution with an explicit confirmation gate, with the runtime holding the commit until the user supplies an approval token. Syll therefore routes by both execution feasibility and the evidence its verification step needs for user review.

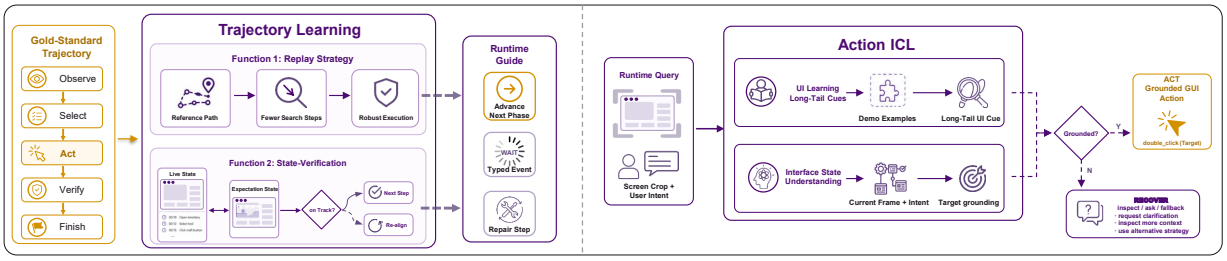
3.2 Demonstration-Teachable Skills and Replay

Some operational knowledge is easier to demonstrate than to describe. Task phases, visual cues, wait conditions, and expected post-action states all live more naturally in a recorded demonstration than in a written prompt, and capturing them is what makes the resulting skill different from a blind coordinate-level replay. Syll stores each desktop demonstration as a reusable runtime artifact, preserving a natural recording interface for users while exposing a state-aware replay representation to the agent through the same unified registry used for text-based skills. Building on programming-by-demonstration and demonstration-guided GUI automation [3, 8, 18, 19], our contribution lies in elevating the demonstration to a first-class runtime abstraction, with the schema (Table 8 in Section D) connecting the recording, replay, and audit subsystems. Once published, a skill becomes accessible via the web UI, invocable through natural language, and schedulable as a proactive routine.

Replay combines phase indexing with in-context action grounding (Action ICL), which packs the previous, current, and next demonstrated steps into the actor's window (Figure 3b). A phase index tracks the active step in the recorded demonstration. At each iteration the packed window conditions the actor's next-action decision on long-tail UI cues such as localized labels, repeated controls, and visually similar icons that are ambiguous from the current screenshot alone. After the executor performs an action, the actor captures the next screenshot and runs the verification step against the current and next expectations. The index advances when either expectation matches, and otherwise holds for recovery. WAIT is emitted when the screenshot is a transient pre-state such as a loading screen, animation, or application launch, and is recorded as a typed executor event so the audit log and action budget distinguish waiting from task progress. RECOVER re-grounds against the same in-context window without advancing the index when no expected predicate matches. Together these affordances preserve state across asynchronous interface transitions where a screenshot-only policy would otherwise repeat completed phases. Algorithm 1 in Section C gives the full replay



(a) Demonstration recording and packaging. The four user-facing steps (*Record*, *Action Slice*, *Build Trace*, *Package Skills*) turn raw screen, mouse, and keyboard events into a reusable skill directory. The internal row below shows the on-disk artifact each step produces.



(b) State-aware replay. *Trajectory learning* reuses the recorded demonstration as a reference path and an expectation source, emitting ADVANCE, WAIT, or repair signals to the runtime guide. *Action ICL* grounds each next GUI action against demonstrated long-tail UI cues, falling back to RECOVER when no target is grounded.

Figure 3. Demonstration-teachable skills in Syll. (a) Recording and packaging. (b) State-aware replay.

episode.

3.3 Document- and Capability-Evolution

Self-evolution in Syll happens at the artifact layer rather than the model-weight layer. Identity, memory, routines, skills, traces, and evidence live as plain editable files instead of as fine-tuning data, refined continuously by both the user and Syll's own writes (Table 9 in Section E enumerates the artifact categories and representative filenames). During context construction, the builder resolves workspace variables and injects the relevant document slices into the prompt, so any modification by the user or the agent changes subsequent behavior. The resulting state is a set of plain files that the user can back up, audit, or delete, not an opaque memory service.

Two loops sit on top of this workspace. The *document loop* lets the user and Syll co-evolve through identity, rules, profile, lore, and routine markdown files. Users refine identity boundaries, communication tone, and behavioral rituals directly in plain text, while Syll updates local memory artifacts as conversations, tool outcomes, and feedback accumulate. The *capability loop* turns runtime interactions into local artifacts the agent can retrieve, execute, inspect, and refine. Traces log execution history through screenshots, tool arguments, and user confirmations. A skill distills a repeatable procedure into a declarative capability the agent can discover and the user can edit, and a routine attaches such a capability to a recurring trigger so the agent acts on time or event signals. Habits therefore accumulate as auditable, time-aware automation rather than hidden model state.

Per-layer evidence and approval gates jointly form Syll's trust boundary across all execution layers, so the agent's past actions are reconstructible and its side-effectful next steps must be confirmed before they commit. Channels, tools, actors, schedules, and skill types are reached through explicit extension boundaries, which lets researchers extend the runtime without rewriting the core executor loop.

The name *Syll* encapsulates this stance. A syllable means little on its own and acquires meaning from the surrounding utterance, and the agent likewise begins with a structured but incomplete identity that gains specificity from accumulated documents.

4 Evaluation

Our evaluation targets the heterogeneous personal-automation setting that Syll's method addresses, where requests cross chat surfaces, local files, command-line tools, browser sessions, desktop GUIs, and scheduled routines. We use three studies. **Study A** probes execution-layer routing when the task does not specify an interface. In **Study B**, we isolate what a recorded demonstration contributes to GUI replay by toggling the demonstration on and off. **Study C** follows a single user across five episodes over a simulated month and tracks how the workspace evolves. Multimodal evidence threads through all three as the shared audit surface. Each run reports intent, route decision, execution trace, produced artifact, and the audit surface itself, with timing, model calls, tokens, screenshots, and estimated cost retained as local efficiency metadata rather than cross-system rankings. Table 3 consolidates this map together with each study's controlled contrast and main readout.

Table 3. Evaluation map. Each study isolates one mechanism and reports a compact auditable readout.

Study	Mechanism isolated	Controlled contrast	Main readout
Study A Execution routing	layer choice across tools, CLI, and visual inspection	file delivery, local command, and screenshot fallback probes	**3/3** route matches; confirmations and audit artifacts retained
Study B Demo replay	effect of a recorded trajectory in clean GUI replay	demo-guided replay vs instruction-only replay	**3/3** vs **2/3** phases; **0** vs **3** off-trace actions
Study C Artifact evolution	persistence, refinement, and routine promotion	Full Syll vs Frozen Workspace across five delayed episodes	**5/5 + 4/4** reuse; **refine=1**; **routine=1**

The studies are complementary. Studies A and B exercise the runtime under model-driven probes with a shared model, executor, and harness. Study C uses a deterministic artifact checker driven by scripted user turns, so its outcomes are independent of how well the model extracts preferences. Aggregate run metadata for the two model-driven studies appear in Table 7.

4.1 Study A on Cross-Surface and Execution-Space Coverage

Study A evaluates whether Syll routes through the appropriate execution layer when the task itself does not specify an interface. We construct probes around side-effectful file delivery, a deterministic local command, and screen-only visual inspection. A run passes when the selected layer matches the intended interface, the task outcome is satisfied, the corresponding audit evidence is retained, and no side-effect error occurs. Run-level timing, calls, tokens, and screenshots are reported alongside but do not enter the pass criterion.

A1 (file delivery) asks Syll to locate a local presentation from a chat surface and select one candidate by visual attribute ("the green one"), then deliver it through a side-effectful channel. Because the delivery is side-effectful, this also tests the confirmation invariant for external sends. The deterministic-command probe **A2** sets up a local computation whose state is machine-readable through a single CLI call, so the test is whether Syll defaults to the command layer rather than to visual control. For the visual-fallback probe **A3**, the relevant state lives only in pixels, exercising the visual inspection layer in isolation without triggering multi-step GUI manipulation.

Table 4. Study A result ledger. The main result is route selection and retained audit evidence. Efficiency metadata are secondary.

Probe	Route	Task outcome	Evidence	Time	Calls	Tokens	Err.
A1 File	structured → confirm	selected and attached the green Aurora roadmap deck	previews, paths, confirmation	25.0 s	6	85.7 k	0
A2 CLI	local CLI	returned `cwd` and top-level entry count without GUI	command log, stdout	4.5 s	2	26.6 k	0
A3 Visual	visual screenshot	captured one screenshot and summarized visible UI state	screenshot, state summary	13.3 s	2	28.9 k	0
Suite	**3/3 match**	**3/3 pass**	**2 confirm gates**	**42.8 s**	**10**	**141.1 k**	**0**

All three runs picked the expected layer and produced the corresponding audit evidence. For **A1**, Syll invoked structured

file tools and previewed the candidates, then blocked delivery until the user supplied a confirmation token. **A2** returned machine-readable workspace state from a single CLI call without touching the GUI. For **A3**, a single screenshot satisfied the inspection and was retained as evidence. Together the runs follow the layered routing policy of Table 2, picking the narrowest viable layer per probe.

4.2 Study B on Demonstration-Trajectory Ablation

Study B isolates the contribution of the recorded user demonstration. We fix the GUI actor, executor, task instruction, and initial interface setup, and vary only whether Syll provides the demonstration skill artifact. The artifact contributes both Action ICL grounding for local UI cues and demonstration-conditioned phase indexing across asynchronous interface transitions, and the contrast therefore probes both at once.

The task requires the agent to launch Stardew Valley from the macOS desktop, open the Load menu, and select a specific farm save (`Vlm/VLMBot`). The workflow stresses parts of replay that are hard to recover from the current screenshot alone. Long-tail UI cues include a Load button, an hourglass save icon, and the player-and-farm save label. The agent must move through launch, load-menu, and farm phases in order, with several loading and animation states absorbed by the typed WAIT/RECOVER loop, and a terminating farm-world HUD gives a deterministic visual success condition.

In **demo-guided replay**, the actor receives the task instruction u, the current screenshot I_k, and the demonstration skill artifact (Table 8) with keyframes kf and semantic traces $trace$. **Instruction-only replay** withholds the artifact, so the actor sees only u and I_k. Any change in phase coverage, off-trace actions, or cue grounding is therefore attributable to the demonstration.

Table 5. Study B clean demonstration-trajectory ablation. Demonstration guidance mainly changes phase indexing and long-tail cue grounding, not the model-call budget.

Evidence item	Demo-guided replay	Instruction-only replay
Result	**Pass**; farm HUD visible, Wed. 3, 6:00 am, 50g	**Fail**; target save not loaded
Phase coverage	**3/3**; launch → Load → farm	**2/3**; save phase missing
Action accounting	**3 semantic actions**; 8 state checks; **0 off-trace**	**15 GUI steps**; **3 off-trace**
Phase indexing	waits through launch and menu loading; **0 order violations**	**10 repeated phase visits**; revisits earlier phases after state changes
Cue grounding	**4/4**; app icon, Load button, save identity, hourglass	**2/4**; app icon and Load button only
Efficiency anchors	**148.2 s**; 8 shots; 16 calls; 43.3 k tokens	**201.1 s**; 15 shots; 17 calls; 43.2 k tokens

Instruction-only replay reproduces the failure mode predicted in Section 3.2. The actor grounded the app icon and the Load button, but exhausted the 15-step budget without selecting the `Vlm/VLMBot` save and revisited completed phases ten times after the interface changed. The failure therefore lies in phase indexing and post-action checking rather than low-level GUI actuation.

Demo-guided replay reaches the final farm state with comparable model-call and token totals (16 vs 17 calls, 43.3 k vs 43.2 k tokens). Action ICL grounds all four long-tail cues, including the hourglass save icon and the player-and-farm save label that instruction-only replay misses. Demonstration-conditioned phase indexing keeps the actor on the demonstrated path with zero off-trace actions and zero phase regressions, while WAIT events absorb the asynchronous launch, menu, and save-list transitions explicitly. The 52.9 s wall-time and 46.7% screenshot reductions follow from these two effects rather than from a direct efficiency objective.

4.3 Study C on Month-Scale Artifact Self-Evolution

Study C evaluates Syll's document-mediated self-evolution claim as inspectable artifact change. The scenario follows a weekly project review workflow over a simulated month. The user teaches the agent formatting preferences and a specific writing style, reuses them later, refines a preference, and eventually promotes the workflow into a scheduled routine. We compare a Full Syll condition that persists artifacts across episodes against a Frozen Workspace baseline that resets them. The sequence is executed by a deterministic artifact checker rather than a live human study, and Syll is credited only for changes that appear as before/after document diffs, approval records, reuse hits, or routine artifacts.

The contrast is between **Full Syll**, where `USER.md`, `SOUL.md`, and routine artifacts persist across episodes, and

Frozen Workspace, where each episode performs the same per-episode extraction but resets artifacts before the next episode begins. The two conditions differ only in artifact persistence, so any difference in later reuse, refinement, or routine promotion is attributable to it.

Table 6. Study C month-scale artifact co-evolution ledger. Metric styling highlights the artifact transition.

Time	Episode	Artifact	Full Syll	Frozen Workspace
Week 1 Day 1	Preference capture	`USER.md`	**+5 USER facts**; Aurora source, format, length, delivery preference; diff saved	captures current-turn facts; reset before next episode
Week 1 Day 3	Style alignment	`SOUL.md`	**+4 SOUL rules**; calm technical voice approved; diff saved	accepts current-turn style patch; reset before next episode
Week 2 Day 8	Joint reuse	both	**reuse 5/5 + 4/4**; 0 clarifications; 39 user words saved	0/5 facts + 0/4 rules retained; 1 clarification
Week 3 Day 15	Preference refinement	`USER.md`	**refine=1**; Blockers made explicit; USER diff saved	missing prior context; 1 clarification; no refinement diff
Week 5 Day 29	Routine promotion	routine	**routine=1**; refined Blockers, voice policy, confirmation retained	no qualified routine; 1 clarification

Both conditions capture preferences and style rules within a single turn, showing that one-shot extraction does not require persistence. The two diverge from Week 2 onward. Only Full Syll reuses the captured artifacts when the user issues the short request "Do my Aurora weekly review." A week later, when the user refines the Blockers preference, only Full Syll has the prior context needed to produce an auditable `USER.md` diff. Frozen Workspace cannot. By Week 5, only Full Syll has accumulated enough state to promote the workflow into a qualified `aurora_weekly_review` routine referencing the Aurora source file, the three-section format, the refined Blockers preference, the `Work-Report Voice` rule, and the draft-first confirmation. The contrast shows that artifact persistence, rather than per-episode model recall, enables later reuse, refinement, and routine promotion.

4.4 Discussion

Scope and limitations. Each study has a narrow scope. The route probes in **Study A** are hand-constructed on one machine, so they validate the selected routes but say nothing about adversarial routing coverage. **Study B** is a single paired replay ($n = 1$ per condition) that exercises the demonstration schema and replay loop, not statistical robustness or environmental perturbation (resolution, theme, language). **Study C** uses a deterministic checker with scripted user turns, so it verifies workspace and routine behavior rather than model extraction quality, population-level preference modeling, or long-term retention. Broader task sets, perturbation studies, and human audit studies are left for future work.

Reproducibility. The release package includes task setup scripts, Study C's deterministic driver, success checks, run-date prompt and provider configuration, workspace layout, JSONL event logs, GUI replay evidence, and the pricing table used for cost fields. API keys, channel credentials, and private user data are excluded. Table 7 reports aggregate metadata for Studies A and B, and Table 4 and Table 5 give the per-run breakdowns.

Table 7. Aggregate run metadata for the model-driven studies A and B. Study C is omitted because its driver is deterministic and not a model-efficiency anchor.

Aggregate	Wall time	Calls	Input tokens	Output tokens	Shots
A-suite (3 probes)	42.8 s	10	140.3 k	0.8 k	4
B-pair (2 runs)	349.3 s	33	86.0 k	0.5 k	23

5 Conclusion

This paper presents Syll as an **open-source multimodal agent harness** for **teachable personal automation**. The core insight is that practical computer use is not only a model problem, but also a problem of **routing** and **translation**. An

effective system must route work across MCP/API tools, CLI execution, and GUI control, while translating human procedures into reusable skills and translating agent behavior back into reviewable multimodal evidence.

Syll implements this view through a self-hosted modular runtime with persistent local artifacts for memory, skills, routines, traces, approvals, and governance. Our mechanism-oriented studies support this design by validating multimodal routing, teachable GUI replay, and persistent local artifacts. Together, these results suggest that open-source agent systems can become more usable and trustworthy when **teachability**, **auditability**, and **extensibility** are treated as first-class system objectives rather than afterthoughts.

Syll still has clear scope limitations. The current evaluation is designed to validate core mechanisms rather than establish large-scale benchmark superiority across diverse operating systems, interface distributions, or user populations. Important next steps include broader task suites, stronger visual grounding verification, longer-horizon memory consolidation, richer MCP/API integrations, and more robust control policies for side-effectful actions. We hope Syll can serve as a practical foundation for user-owned personal automation systems that people can teach, inspect, and continuously extend.

References

[1] Reyna Abhyankar, Qi Qi, and Yiying Zhang. Osworld-human: Benchmarking the efficiency of computer-use agents. *arXiv*, abs/2506.16042, 2025.

[2] Anthropic. Computer use tool. https://docs.anthropic.com/en/docs/build-with-claude/computer-use, 2026. Accessed 2026-04-24.

[3] Allen Cypher. Eager: Programming repetitive tasks by example. In *Proceedings of the SIGCHI conference on Human factors in computing systems*, pages 33–39, 1991.

[4] HKUDS. nanobot: An ultra-lightweight personal ai agent framework. https://github.com/HKUDS/nanobot, 2026. Accessed 2026-04-29.

[5] Leslie Pack Kaelbling, Michael L Littman, and Anthony R Cassandra. Planning and acting in partially observable stochastic domains. *Artificial intelligence*, 101(1-2):99–134, 1998.

[6] Jeonghye Kim, Sojeong Rhee, Minbeom Kim, Dohyung Kim, Sangmook Lee, Youngchul Sung, and Kyomin Jung. Reflact: World-grounded decision making in llm agents via goal-state reflection. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 33421–33453, 2025.

[7] Thomas Kwa, Ben West, Joel Becker, Amy Deng, Katharyn Garcia, Max Hasin, Sami Jawhar, Megan Kinniment, Nate Rush, Sydney Von Arx, et al. Measuring ai ability to complete long software tasks. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.

[8] Gilly Leshed, Eben M Haber, Tara Matthews, and Tessa Lau. Coscripter: automating & sharing how-to knowledge in the enterprise. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, pages 1719–1728, 2008.

[9] Model Context Protocol. What is the model context protocol? https://modelcontextprotocol.io/docs/getting-started/intro, 2026. Accessed 2026-04-24.

[10] OpenAI. Harness engineering: Leveraging codex in an agent-first world. https://openai.com/index/harness-engineering/, 2026. Published 2026-02-11, accessed 2026-04-29.

[11] OpenClaw Contributors. OpenClaw: Open-source self-hosted ai agent framework. https://github.com/openclaw/openclaw, 2025. Accessed 2026-04-29.

[12] Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, et al. Ui-tars: Pioneering automated gui interaction with native agents. *arXiv preprint arXiv:2501.12326*, 2025.

[13] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36:68539–68551, 2023.

[14] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023.

[15] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh J Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *Advances in Neural Information Processing Systems*, 37:52040–52094, 2024.

[16] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.

[17] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.

[18] Tom Yeh, Tsung-Hsiang Chang, and Robert C Miller. Sikuli: using gui screenshots for search and automation. In *Proceedings of the 22nd annual ACM symposium on User interface software and technology*, pages 183–192, 2009.

[19] Yichun Zhang, Xiangwu Guo, Yauhong Goh, Jessica Hu, Zhiheng Chen, Xin Wang, Difei Gao, and Mike Zheng Shou. Showui-aloha: Human-taught gui agent. *arXiv preprint arXiv:2601.07181*, 2026.

[20] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, 2024.

Supplementary Material

Contents

A Author Information

We provide author contributions and contact information together in this appendix as follows:

- **Borui Zhang**: Project leader of Syll; coordinated the overall research effort, guided the project direction, and contributed to code development.
  Contact: `zhang-br21@mails.tsinghua.edu.cn`
- **Bo Zhang**: Led the framework design, was primarily responsible for most of the system framework construction, and took the main role in code implementation and maintenance.
  Contact: `bo-zhang23@mails.tsinghua.edu.cn`
- **Chenghao Jiang**: Mainly designed the multimodal GUI grounding and localization tools and contributed to code development.
  Contact: `chenghao.jiang2022@outlook.com`
- **Minglei Shi**: Participated in project discussions, contributed constructive suggestions, and contributed to code and feature development.
  Contact: `sml25@mails.tsinghua.edu.cn`
- **Jie Zhou**: Provided overall guidance and supported the project with resources.
  Contact: `jzhou@tsinghua.edu.cn`
- **Jiwen Lu**: Corresponding author; provided overall guidance and supported the project with resources.
  Contact: `lujiwen@tsinghua.edu.cn`

B Additional Implementation Details

The implementation is partitioned across `syll/agent/` for the core loop, context, skills, GUI planning, and tools; `syll/channels/` and `syll/bus/` for adapters and message flow; `syll/cron/` for scheduled routines; `syll/recorder/` for desktop recording; and `syll/web/` for FastAPI routes and the browser UI.

C Formal Model

Typed artifact-option process. We model Syll as a typed artifact-option process rather than a single reasoning-action transcript. The joint state contains both the external computer state and the agent-owned artifact state:

$$S_t = (X_t, A_t).$$

A channel event is a partial observation of this state. Writing $\xi_t = (c_t, e_t)$ for the observed channel and raw event,

$$\xi_t \sim O(\cdot \mid S_t), \qquad m_t = \mathrm{Adapter}_{c_t}(e_t).$$

The context builder then constructs a typed, task-relevant projection

$$C_t = \Phi(A_t, m_t) = (W_t, H_t, R_t, T_t, V_t, P_t).$$

Because the true computer state is partially observable and the prompt is finite, C_t is a lossy projection of the belief, not a sufficient statistic. The policy first chooses an execution route and then emits a typed action:

$$r_t \sim \pi_\theta^R(\cdot \mid C_t), \qquad (\alpha_t, \hat{\psi}_t, \chi_t) \sim \pi_\theta^A(\cdot \mid C_t, r_t),$$

with $a_t = (r_t, \alpha_t, \hat{\psi}_t, \chi_t)$. Here χ_t denotes the confirmation requirement emitted with the action, such as no gate, preview required, or explicit user approval; it is not a model confidence score and is retained as part of the audit record. Execution returns both an observation and an evidence artifact, after which the artifact state is updated:

$$(o_{t+1}, z_{t+1}) = \mathrm{Exec}_{r_t}(\alpha_t, X_t), \qquad A_{t+1} = U_A(A_t, m_t, a_t, o_{t+1}, z_{t+1}, \hat{\psi}_t).$$

Setup and observations. Let $X_t = (X_t^{\mathrm{os}}, X_t^{\mathrm{app}}, X_t^{\mathrm{web}}, X_t^{\mathrm{file}}, X_t^{\mathrm{user}}, G_t)$ be the partially observed computer state and let $A_t = (W_t, H_t, R_t, T_t, V_t, P_t, \Gamma_t)$ be the agent-owned artifact state. The joint state is $S_t = (X_t, A_t)$ and the observable history before the current decision is $h_t = (m_{1:t}, a_{1:t-1}, o_{1:t-1}, z_{1:t-1})$, with belief $b_t(S) = \mathbb{P}(S_t = S \mid h_t)$. Channel events are observations of this state:

$$\xi_t = (c_t, e_t) \sim O(\cdot \mid S_t), \qquad m_t = \mathrm{Adapter}_{c_t}(e_t).$$

Process dynamics. The context builder is a lossy prompt projection, not a sufficient statistic:

$$\Phi : \mathcal{A} \times \mathcal{M} \to \mathcal{C}, \qquad C_t = \Phi(A_t, m_t) = (W_t, H_t, R_t, T_t, V_t, P_t).$$

The policy and executor decompose decisions into route choice, concrete action, expected postcondition, confirmation status, observation, and evidence:

$$r_t \sim \pi_\theta^R(\cdot \mid C_t), \qquad (\alpha_t, \hat{\psi}_t, \chi_t) \sim \pi_\theta^A(\cdot \mid C_t, r_t),$$

$$(o_{t+1}, z_{t+1}) = \mathrm{Exec}_{r_t}(\alpha_t, X_t), \qquad A_{t+1} = U_A(A_t, m_t, a_t, o_{t+1}, z_{t+1}, \hat{\psi}_t).$$

Skills as options. A full skill is $s = (\mu_s, \Omega_s, \pi_s, \beta_s, \Psi_s, \Gamma_s)$, an option with metadata, initiation predicate, intra-skill policy, derived termination, postconditions, and evidence schema. A demonstration induces $D_s = \{(I_i, \ell_i, \alpha_i, \psi_i, z_i)\}_{i=1}^n$ and terminates by

$$\beta_s(S_t) = \mathbf{1}[\mathrm{Match}(I_t, \Psi_s^{\mathrm{terminal}})], \qquad \Psi_s^{\mathrm{terminal}} = meta.\mathrm{success\_check}.$$

Replay maintains $q_t(i) = \mathbb{P}(p_t = i \mid \hat{\tau}_{\le t}, D_s)$ with monotone kernel $P_{\mathrm{phase}}(i \mid j)$ that assigns $1 - p_{\mathrm{adv}}$ to $i = j$, p_{adv} to $i = j + 1$, and 0 otherwise, retaining boundary mass at the terminal phase. The update and marginal policy are

$$q_t(i) \propto \mathrm{Match}(I_t, \psi_i) \sum_j P_{\mathrm{phase}}(i \mid j) q_{t-1}(j),$$

$$\pi_s(a \mid C_t, D_s) = \sum_i q_t(i) \big[\lambda \delta_{\alpha_i}(a) + (1 - \lambda) \pi_\theta(a \mid I_t, \ell_i, \psi_i)\big].$$

Algorithm 1 is the MAP implementation: it stores only $p = \arg\max_i q_t(i)$. Composition is admissible when $\mathrm{Entails}_A(\Psi_{s_1}^{\mathrm{terminal}}, \Omega_{s_2}) = 1$.

ReAct as a degenerate case. The ReAct family interleaves free-form reasoning traces with environment actions [17]; Reflexion and ReflAct add verbal feedback and goal-state reflection [6, 14]. It is recovered by taking a single channel, collapsing A_t to a textual transcript H_t, removing persistent evidence artifacts and skills, and fixing the route to the environment:

$$(\tau_t, \alpha_t) \sim \pi_\theta(\cdot \mid H_t).$$

Thus ReAct is the monolithic-text, no-option, no-audit special case.

Algorithm 1 Demonstration-conditioned replay (one episode).

Require: Skill $s = (meta, traj, kf, trace)$ with N semantic steps; instruction u; context C_t; step budget K; wait interval δ

$p \leftarrow 0$; evidence $E \leftarrow \emptyset$ {phase index, audit log}
for $k = 1$ to K **do**
  $I_k \leftarrow \text{CaptureScreenshot}()$; $E \leftarrow E \cup \{I_k\}$
  if $\text{SuccessCheck}(I_k,\ meta.\text{success\_check})$ **then**
    return $\langle \text{SUCCESS},\ E \rangle$
  end if
  $W \leftarrow [\max(0, p-1),\ \min(N-1, p+1)]$ {phase-positional window}
  $E_k \leftarrow \{(kf[i],\ trace[i]) : i \in W\}$ {Action ICL pack}
  $r \leftarrow \text{Actor}(C_t,\ u,\ E_k,\ p,\ I_k)$ $\{r.\text{kind} \in \{\text{ACT}, \text{WAIT}, \text{RECOVER}\}\}$
  if $r.\text{kind} = \text{WAIT}$ **then**
    $\text{Sleep}(\delta)$; $E \leftarrow E \cup \{r\}$; **continue**
  end if
  if $r.\text{kind} = \text{RECOVER}$ **then**
    $E \leftarrow E \cup \{r\}$; **continue** {re-ground next iteration; p unchanged}
  end if
  $\text{DesktopExecutor}(r.\text{action})$; $E \leftarrow E \cup \{r.\text{action}\}$
  $I_{k+1} \leftarrow \text{CaptureScreenshot}()$
  if $\text{Match}(I_{k+1},\ trace[p].\text{expectation})$ **or** $(p{+}1 < N$ **and** $\text{Match}(I_{k+1},\ trace[p{+}1].\text{expectation}))$ **then**
    $p \leftarrow p + 1$ {advance, possibly through a transient state}
  end if
end for

return $\langle \text{BOUNDED-FAILURE},\ E \rangle$

C.1 Optimization with Auditability

Auditability is modeled as a constraint rather than a reward bonus:

$$\max_{\pi} \mathbb{E}_\pi \left[\sum_{t=0}^{T} \gamma^t R(S_t, a_t) \right] \quad \text{s.t.} \quad \mathbb{E}_\pi[\text{Coverage}(z_{1:T}, \Psi)] \geq \kappa, \qquad a_t \in \mathcal{A}_{\text{side}} \Rightarrow \chi_t = 1.$$

The Lagrangian form subtracts $\lambda(\kappa - \mathbb{E}[\text{Coverage}(z_{1:T}, \Psi)])$, while the runtime enforces the constraint directly through confirmations, logs, screenshots, keyframes, and diffs.

Propositions and assumptions. **P1: Markov augmentation.** Given P_X, O, and U_A, the augmented process over $S_t = (X_t, A_t)$ is Markov before the current event is observed: X_{t+1} depends on X_t and a_t, while A_{t+1} depends on A_t and the current message, action, observation, evidence, and expected postcondition. **Assumption 1: artifact-summary ideal.** The artifact updater is designed to preserve history features needed for future action selection; in practice $C_t = \Phi(A_t, m_t)$ is lossy. Under this ideal, the standard belief-MDP reduction applies [5]. **P2: demonstration as option.** D_s induces an option with initiation predicate Ω_s, intra-option policy π_s, and terminal-check termination β_s; therefore it is a semi-MDP action. **P3: ReAct degeneracy.** Under the degenerate setting above, the typed artifact-option process reduces to ReAct because the only remaining decision is emitting a reasoning trace and an environment action from the textual transcript.

D Demonstration-Skill Schema

Each demonstration skill referenced in Section 3.2 is encapsulated as a self-contained directory whose fields parameterize the replay loop and the algorithmic episode in Algorithm 1. The metadata holds the terminal success predicate that the

verification step evaluates at the end of replay; the trajectory holds the deterministic-fallback event sequence dispatched by the executor; the keyframes anchor Action ICL retrieval at each step; and the trace gives per-step expectations used by the verification step as the phase index advances.

Table 8. Demonstration-skill schema. Each field has a concrete replay role.

Field	Stored content	Replay and audit use
meta	Name, app context, aliases, preferred replay mode, terminal `success_check` predicate	Registry summary; channel dispatch; terminal check in Algorithm 1
traj	Ordered raw events: `click`, `type`, `key`, `scroll`, `wait`, `hotkey`	Desktop executor input; deterministic fallback replay
kf	One screenshot per semantic step in the demonstration	Action ICL image anchor; human-readable audit evidence
trace	Per-step observation, intent, action, and a post-state expectation predicate	Phase indexing; actor prompt construction; per-step verification

E Companion Workspace Layout

Syll stores its companion state as a directory of editable local artifacts. Table 9 enumerates the six artifact categories referenced from Section 3.3; the context builder resolves the relevant slices and injects them into C_t during prompt construction, and both user edits and Syll's writes flow back into the same files.

Table 9. Companion workspace layout. Each row is a directory of editable local artifacts that the context builder draws from when constructing C_t; the document loop and capability loop in Section 3.3 update them through user edits and Syll's writes.

Category	Representative files	Role
Identity & Rules	`IDENTITY.md`, `SOUL.md`, `AGENTS.md`, `TOOLS.md`	Persona, operating constraints, behavioural rules, registered tools
Profile & Memory	`USER.md`, `memory/MEMORY.md`, `daily_notes/`	User-specific facts and accumulating conversation memory
Lore & Rituals	`lore/fragments.md`, `lore/rituals.md`	Long-term relationship context and proactive behavioural rituals
Skill Registry	`skills/`, `gui_skills/`, `aloha_skills/`	Text-based and demonstration-based reusable skills
Traces & Evidence	`event_logs/`, `trajectory.json`, `keyframes/`	Per-run audit trail consumed by the verification step
Schedules	`cron/jobs.json`	Time- or event-triggered routines

F Cross-Surface Runtime and Browser-Native Control

Cross-surface continuity begins at the runtime. Every user-facing surface enters the same local executor loop and shares the same workspace context, so the agent operates over a single ongoing state rather than a set of disjoint applications. Syll realizes this through a unified channel-bus-agent-tool architecture (Figure 2). Channel adapters normalize inputs from diverse sources (web consoles, CLIs, messaging apps, and scheduled jobs) into a standardized message format, and an in-process bus coordinates inbound messages, proactive triggers, and tool observations on behalf of the executor. Regardless of the origin surface, the executor loads the relevant session, constructs the context C_t, queries the model, dispatches validated tool calls, and appends the resulting observations and audit artifacts to the session history.

The browser-native control surface is integral to the system. Through the web UI, users can inspect and edit core artifacts, including model configurations, identity documents, skills, and activity logs. Unlike developer-oriented gateways such as OpenClaw [11], Syll treats browser-based interaction as a first-class runtime surface that keeps the agent's internal state visible and editable for non-expert users. Beyond serving end users, this architecture is designed for research and community extension: the core executor loop is decoupled from peripheral interfaces, so channels, tools, and skills interact through standardized boundaries and plain-text persistence rather than tightly coupled code, letting researchers isolate components, integrate new capabilities, and fork the system for custom experiments.

G Limitations

Syll inherits the main limitations of current agent systems. GUI execution can fail because of visual grounding errors, high-DPI coordinate mismatch, hidden state, modal dialogs, permissions, network delays, or application updates. The demonstration loop can reduce repeated effort, but a single recording is not a guarantee of robust generalization across all UI variants.

The current artifact also has evaluation gaps. The public demos show plausible workflows, but the project still needs controlled quantitative results against strong baselines. Human-friendly auditability is a core claim, so it should be measured with user studies rather than inferred from design alone.

Finally, self-hosting gives users control but also shifts responsibility to local configuration. API keys, channel credentials, desktop permissions, filesystem access, and GUI automation privileges must be handled carefully. For safety, destructive file operations, external messaging, purchases, account changes, or other side effects should require explicit confirmation and clear logs.

H Reproducibility Statement

Syll is released as an open-source Python package and repository. The package, CLI, and import path are `syll`. The current codebase uses Python 3.11+ and includes FastAPI web routes, Typer CLI commands, LiteLLM model access, PyQt-based desktop companion support, channel adapters, GUI tools, recorder modules, and tests for web routes, tool validation, GUI planning, voice routes, cron behavior, and dashboard integration.

To reproduce the artifact-level setup:

- install with `python -m pip install syll` or from source with `pip install -e ".[dev]"`;
- run `syll onboard` to create the local configuration and workspace;
- configure the chat/planner/actor model endpoints and any channel credentials in the local config;
- run `syll wake` for the server and web UI, and optionally `syll ghost` for the desktop companion process;
- run `pytest` and `ruff check syll/` for implementation checks.

For future benchmark results, each reported run should include the model versions, provider endpoints, operating system, display resolution, selected screen, GUI actor mode, maximum steps, channel configuration, random seeds where applicable, and full execution traces with screenshots or videos sufficient to audit success and failure cases.